Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers

Ahmed Gouda Ismail[1*], Mohamed S. Mohamed[1] and Tarek Ahmed Mahmoud[2]

[1*]Computer Engineering and Artificial Intelligence Department, Military Technical College, Cairo, Egypt.

[2]Computer Engineering Department, Egypt University of Informatics, Cairo, Egypt.

*Corresponding author(s). E-mail(s): a7medgouda1@gmail.com
Contributing authors: mohamedms@mtc.edu.eg; tarek.mohamed@eui.edu.eg

Abstract

Multi-object tracking systems combine a detector, which finds objects and gives each a confidence score, with a tracker, which links them over time using fixed score thresholds. When the detector is replaced, retrained, or compressed, its scores change, but the tracker keeps its old thresholds. This paper proposes a training-free self-calibrating control layer (SCL) placed between an unchanged detector and tracker (the host). It splits the scores of recent frames with Otsu's histogram threshold and labels their distribution as clean or noisy. On clean scores it keeps the host's settings; on noisy scores it chooses which detections reach the host, rescores them, and sets its thresholds. It also continues tracks through low-score frames for single-stage trackers. It needs no labels or future frames. One fixed configuration was tested on seven published trackers, four detection sources, and three datasets. All gains below are relative changes over the tracker alone. SCL raises the higher order tracking accuracy (HOTA) of two single-stage trackers by 0.9%: OC-SORT, used during development, and PD-SORT, which was not. With halved detector scores, two trackers failed; with SCL, both recovered at least 97% of their original HOTA. With a noisy RT-DETR-L detector, SCL raises the multiple object tracking accuracy (MOTA) of ByteTrack and BoT-SORT by 12.6% and 16.3% and reduces their identity switches by 43% and 47%. On four trackers that already fit their detections, HOTA changed by less than 0.1%. Paired bootstrap analysis supports these results. SCL adds 2.95 milliseconds per frame on an ordinary processor.

Keywords: multi-object tracking, tracking by detection, detector confidence calibration, online adaptation, Otsu thresholding, real-time image processing

1 Introduction

Multi-object tracking (MOT) finds every object of interest in a video, such as people or cars, and gives each object one identity number (ID) that stays the same over time. It is used in surveillance, driving, and drone systems [1, 2], and many of these systems must give results with little delay, so the processing cost matters as much as the accuracy.

The performance of a MOT system depends on several factors, including the quality of object detection, the ability to associate objects across consecutive frames, and the fit between the two. These factors are closely related, since the detections decide what the tracker can follow. Missed

detections may interrupt object trajectories, while false detections can lead to incorrect associations and identity switches. Achieving a suitable balance between finding the objects and keeping their identities is therefore an important challenge in MOT systems.

Most MOT systems follow the tracking-by-detection design [3]. First, an object detector, a neural network, looks at each frame and returns candidate boxes. Each box has a confidence score between 0 and 1 that says how sure the detector is that the box holds an object. Then a tracker links the boxes of consecutive frames into tracks, one track per object [4–6]. The tracker uses a box only if its score passes a fixed limit, called a threshold. A two-stage tracker, such as ByteTrack, first matches the high-score boxes to its tracks and then uses lower-score boxes in a second step, only to continue tracks that are still unmatched [5]. A single-stage tracker, such as OC-SORT, uses one threshold and ignores every box below it [6]. Table 1 explains the main terms of this paper in plain words.

In real systems, the detector and the tracker are rarely built together. A detector is replaced by a faster model, retrained for a new camera, or made smaller (quantized) for embedded hardware. The tracker, however, keeps the thresholds that its authors set for another detector. Detector scores are not calibrated across models and conditions [7]: the same object can get a high score from one detector and a much lower score from another. A new detector therefore usually gives scores on a different scale, and nothing in the system notices the change. The effect can be large. In this paper, halving the scores of a published detector reduces two well-known trackers to 0 higher order tracking accuracy (HOTA) [8], the main tracking metric of this paper, because no detection passes their fixed thresholds (Sect. 5.3). Retuning every tracker for every detector needs labels from the new setting, that is, boxes and identities drawn by people, and a working system usually does not have them.

Existing adaptive methods only partly address this problem. Trackers with adaptive thresholds change their own thresholds from the detections of each frame, but each method is built into one tracker [9–11]. Score calibration methods correct the detector scores, but they need labelled data [7]. Methods that adapt a vision system while it runs learn or measure their settings for one model [12, 13]. What is missing is a simple, training-free component that works outside the tracker, adjusts itself from the detector's own scores, and leaves the tracker alone when it already fits those scores.

This paper presents such a component: a self-calibrating control layer (SCL) that sits between a detector and a tracker that are both kept fixed (frozen): their code, learned weights, and stored settings are not changed. We call the tracker that SCL serves the *host*, because SCL sits in front of it and works for it without changing its code. The host gives SCL four numbers from its published settings, which we call the host contract (Sect. 3.5); SCL may give the host new values of these thresholds for each frame. The central idea is that the distribution of the detector's own scores gives a label-free indication of whether they fit the host's thresholds. SCL looks at the scores of recent frames and checks how many of the higher scores form a clearly confident group. This describes the scores, not the accuracy of the detections: the layer cannot tell whether a confident box is a real object. If most of the higher scores are confident, the host keeps its own settings, and the layer only adds help that the host lacks, such as a step that continues tracks. If many are uncertain, SCL chooses which candidates reach the host, gives them new scores, and sets the host's thresholds for the frame. What is new is therefore not a detector or a tracker, but a layer outside the tracker that needs no labels, uses one fixed configuration for many trackers, and stays out of the way when the tracker already fits the detector.

The results are consistent across trackers, detectors, and data. With halved detector scores, SCL restored two trackers from 0 to about 66 HOTA. It improved every single-stage tracker that was tested, including one that was not seen during design, and it raised HOTA on every MOT17 sequence for both of them. With other detectors, it raised the multiple object tracking accuracy (MOTA), a metric that counts errors, of two-stage trackers by up to 7.9 points on KITTI and by 24.7 points on held-out aerial data, that is, data that was not used during the design. It also cut the identity switches, the cases where an object's ID changes, by 43% and 47% for the two-stage trackers on KITTI and by 45% and 56% on the aerial data. On the same scores, it reached higher HOTA than simple adaptive thresholds by 2.2 to 8.6 points, and it was ahead of the

Table 1 Main terms of this paper in plain words.

Term	Meaning in this paper
Detector	A neural network that finds objects in one video frame and returns a box for each of them.
Candidate (detection)	One box returned by the detector, with a class label (for example, car) and a confidence score.
Confidence score	A number between 0 and 1 that says how sure the detector is that the box holds an object. Different detectors use different score scales.
Threshold	A score limit in the tracker's settings. The tracker uses a candidate only if its score is at or above the limit.
Tracker	A program that links the candidates of consecutive frames into tracks, one track per object.
Track, identity (ID)	The boxes of one object over time, with one ID number that should never change.
Sequence	One video of a dataset.
Stream	The detector output of one video, frame after frame.
Host	The tracker that SCL serves. It runs after SCL with its own published code and settings.
Host contract	Four published settings of the host that SCL reads (Sect. 3.5).
Matching and birth thresholds	The score a candidate needs to continue an existing track (matching, also called association), and the score it needs to start a new track (birth).
Single-stage, two-stage tracker	A single-stage tracker uses one threshold and ignores every candidate below it. A two-stage tracker also uses low-score candidates in a second step, only to continue existing tracks.
Frozen	Kept fixed: the code, the learned weights, and the stored settings are not changed. For a frozen host, SCL may still pass new values of its contract thresholds for each frame, through the tracker's own parameters; its code and stored settings stay the same. A frozen SCL is one fixed configuration used for every test.
Labels (ground truth)	Boxes and identities drawn by people. They are used only to measure the results, never by SCL.
Development data	Data that was looked at while SCL was designed.
Held-out, predeclared	Data or trackers that were chosen in advance, were not used during the design, and were run once with the frozen SCL.
Logit	The score written on another scale, $\log\big(s/(1-s)\big)$, Eq. (10). It spreads out scores close to 0 and 1.
Otsu's method	A standard image-processing rule that picks the threshold that best splits a histogram into two groups [14].
Clean, noisy stream	Two kinds of score distribution: clean when most foreground candidates have clearly high scores, noisy when many of them are uncertain. They describe the scores, not whether the boxes are correct.
HOTA, MOTA, IDF1	Tracking metrics in percent; higher is better (Sect. 2.2). HOTA, the main metric, balances finding the objects and keeping their identities.
FP, FN, IDS	False positives (boxes where there is no object), false negatives (missed objects), and identity switches (an object's ID changes). They are counts; lower is better.
Paired comparison	The same tracker runs twice on the same sequences and detections, once alone and once with SCL.
95% interval, significant	The range of likely values of a difference, found by resampling the test sequences 10,000 times (bootstrap). A difference is called significant when this range does not include zero.

published adaptive-threshold method ACT under halved scores (by 2.0 and 10.1 HOTA points) and on held-out aerial data (by 1.8 HOTA points with YOLOv8n and 22.7 MOTA points with RT-DETR-L). The tested strong trackers that already fit their detector stayed within 0.06 HOTA points of their own accuracy, and no tested combination of data, detector, and tracker showed a statistically significant HOTA loss.

The main contributions are:

1. SCL, a training-free and causal control layer that adjusts itself from the detector scores, keeps the tracker's settings when the scores are clean, and takes full control only when they are noisy, together with a host contract of four published settings that lets one fixed configuration work with single-stage and two-stage trackers without changing them (Sect. 3).
2. A development path from always-on banding (AOB), an earlier design that always intervened, to the final layer, in which each step is compared with the step it was built on (Sect. 4).
3. An evaluation of one frozen configuration on seven published trackers, four detection sources, and three datasets, with paired bootstrap intervals, a comparison with simple adaptive thresholds and with the published ACT method, and an analysis of each part of the layer (Sect. 5).
4. A latency measurement showing that the layer adds only about 3 ms per frame on an ordinary processor (central processing unit, CPU) (Sect. 3.7).

The rest of the paper is organized as follows. Section 2 describes the datasets and evaluation metrics and reviews related work. Section 3 specifies SCL with all its values, and Section 4 explains

how it was developed from an earlier design that always intervened. The final configuration was then frozen before any external system was run. Section 5 gives the experimental settings and the results, and Section 6 a discussion. Section 7 concludes the paper.

2 Background and Related Work

This section introduces the datasets and evaluation metrics used in this study, followed by a review of related work on MOT.

2.1 Dataset Description

Three publicly available video datasets, MOT17 [15], KITTI tracking [16], and UAVDT [17], are used to develop and evaluate the proposed SCL. They differ in their objects, their cameras, and their recording conditions, and they are used with different detectors. Using these datasets allows us to examine SCL with well-matched detections, with other detectors, and on aerial videos that were not used during the design. Table 2 lists the evaluation cells; a cell is one combination of data, detector, and tracker.

2.1.1 MOT17

MOT17 is a widely used benchmark for tracking people [15]. It contains 14 video sequences, 7 for training and 7 for testing, filmed in public places with static and moving cameras. As is common, each of the seven training videos is split in time, and the second half, called val-half, is used for evaluation [5]; it has 2,652 frames. On MOT17 the trackers use the published detections of the YOLOX-X detector [18], so every tracker sees exactly the same boxes. A detector drops the boxes whose score is below a small value before it passes them on; we call this value the emission floor. The two published MOT17 detection sets have floors of 0.01 and 0.1. These detections fit the published trackers well, so MOT17 shows whether SCL leaves a well-matched tracker unchanged.

To study the score problem directly, the published MOT17 scores were also changed before the tracker by four monotone transforms, which change the score values but keep their order: cubing (s^3), halving ($0.5\,s$), and logit temperatures $T = 2$ and $T = 0.5$, $\sigma(z/T)$, where σ turns a logit back into a score. A temperature divides the logit z of each score by T before turning it back into a score: $T = 2$ pulls all scores toward 0.5, and $T = 0.5$ pushes them toward 0 and 1. These transforms imitate the score changes that a new or retrained detector can cause. The trackers kept their published thresholds.

2.1.2 KITTI tracking

KITTI tracking is a benchmark recorded from a car that drives in and around Karlsruhe, Germany [16]. Its training set has 21 sequences with 8,008 frames, and its official evaluation scores cars and pedestrians. In this study the trackers run on KITTI with two detectors that differ from the MOT17 detector: YOLOv8n [19], a small and fast model, and RT-DETR-L [20], a larger transformer model. The trackers keep the thresholds that were set for other detectors, so KITTI shows what happens when the detector is replaced.

2.1.3 UAVDT

UAVDT is a benchmark designed for object detection and MOT in videos from unmanned aerial vehicles (UAVs), with a particular focus on vehicles [17]. It consists of 100 video sequences selected from more than 10 hours of video recorded at urban locations such as squares, arterial streets, toll stations, highways, crossings, and T-junctions, at 30 frames per second and a resolution of 1080×540 pixels. The sequences are recorded under different weather conditions, flying altitudes, and camera viewing angles. The test set contains 20 sequences for the MOT task, with 16,592 frames, and these 20 sequences are used in this study. They were not used during the design, and ByteTrack runs on them with YOLOv8n and RT-DETR-L, restricted to their vehicle classes.

The datasets serve different purposes in the evaluation. Development cells were looked at while SCL was designed: MOT17 with four trackers, KITTI with two other detectors, and the score transforms. The other cells test SCL on trackers and data that it had never seen. The two predeclared external trackers, PD-SORT and Hybrid-SORT, were chosen and named before they were run, and they were run once with the frozen configuration on the detections released

by their authors on MOT17 val-half. The held-out UAVDT test split was also run once with the frozen configuration, after a written plan. To check that each external tracker was run correctly, our run of the tracker alone is compared with the number reported by its authors and marked EXACT (within 0.2 HOTA points) or CLOSE (within 1.0 HOTA point, with a known cause that was not tuned).

2.2 MOT Evaluation Metrics

To evaluate SCL, six commonly used MOT metrics are considered: Higher Order Tracking Accuracy (HOTA), Multiple Object Tracking Accuracy (MOTA), Identity F1 Score (IDF1), identity switches (IDS), false positives (FP), and false negatives (FN). These metrics measure different aspects of tracking performance, including detection accuracy, object identity consistency, and the ability to track objects across consecutive frames [8, 21]. They are defined below, starting from the errors counted in each frame and ending with the scores that combine them.

Detection errors. The first metrics count the errors made frame by frame. In each frame t, output boxes are matched one to one to ground-truth boxes with an intersection over union (IoU) of at least 0.5. Let GT_t be the number of ground-truth boxes, TP_t the number of matched pairs, FP_t the number of output boxes without a match (false positives), and FN_t the number of ground-truth boxes without a match (false negatives, or missed objects). Over all frames,

$$\mathrm{GT} = \sum_t \mathrm{GT}_t, \qquad \mathrm{TP} = \sum_t \mathrm{TP}_t, \tag{1}$$

$$\mathrm{FP} = \sum_t \mathrm{FP}_t, \qquad \mathrm{FN} = \sum_t \mathrm{FN}_t, \tag{2}$$

so that $\mathrm{TP} = \mathrm{GT} - \mathrm{FN}$.

Identity switches. The counts above treat each frame separately, so they do not show whether an object keeps the same track over time. Identity switches measure this. Let $\phi_t(i)$ be the output track matched to ground-truth object i in frame t, and $\phi_{\mathrm{prev}}(i)$ the output track to which object i was last matched before t. An identity switch is counted whenever these two tracks differ:

$$\mathrm{IDS} = \sum_t \sum_i \mathbf{1}[\phi_t(i) \neq \phi_{\mathrm{prev}}(i)], \tag{3}$$

where the inner sum runs over the objects that are matched in frame t and were matched before. A tracker that follows more objects has more chances to switch identities, so IDS is read together with FN.

MOTA and IDF1. These two scores summarize the errors above in a single number each. MOTA combines the three error types [22], and IDF1 measures how long each object keeps the correct identity [23]:

$$\mathrm{MOTA} = 1 - \frac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}, \tag{4}$$

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}. \tag{5}$$

For IDF1, each ground-truth track is first matched to at most one output track over the whole video. IDTP, IDFP, and IDFN are then the true-positive, false-positive, and false-negative boxes under this track-level matching. Because MOTA counts every false positive and every missed object, it reacts strongly to how many boxes reach the tracker. IDF1 depends more on whether each object keeps one identity.

HOTA. MOTA depends mostly on detection errors, and IDF1 mostly on identity errors. HOTA balances detection and association in one score [8], and it is the main metric of this paper. At a localization threshold α, let TP_α, FN_α, and FP_α be the matched pairs, missed ground-truth boxes, and unmatched output boxes. For a matched pair c, $\mathrm{TPA}(c)$ is the set of matched pairs with the same ground-truth identity and the same output identity as c; $\mathrm{FNA}(c)$ is the set of boxes of the same ground-truth object that are matched to another output identity or not matched; and $\mathrm{FPA}(c)$ is the set of boxes of the same output track that are matched to another object or not matched. Then

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}_\alpha|}{|\mathrm{TP}_\alpha| + |\mathrm{FN}_\alpha| + |\mathrm{FP}_\alpha|}, \tag{6}$$

$$\mathcal{A}(c) = \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|}, \tag{7}$$

$$\mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}_\alpha|} \sum_{c \in \mathrm{TP}_\alpha} \mathcal{A}(c), \tag{8}$$

Table 2 Evaluation cells and their role. A cell is one combination of data, detector, and tracker. Development cells were looked at while the layer was designed; predeclared and held-out cells were chosen in advance and run once with the frozen layer.

Data	Detector	Trackers	Role
MOT17 val-half (7 sequences)	published YOLOX-X detections	ByteTrack (2 settings), OC-SORT, BoostTrack, SparseTrack	development
KITTI tracking training (21 sequences)	RT-DETR-L; YOLOv8n	ByteTrack and BoT-SORT (RT-DETR-L); OC-SORT (YOLOv8n)	development (detector transfer)
MOT17 val-half, score transforms	published YOLOX-X, transformed scores	ByteTrack (2 settings), OC-SORT, BoostTrack	development (robustness)
MOT17 val-half	released PD-SORT / Hybrid-SORT detections	PD-SORT, Hybrid-SORT	predeclared external
UAVDT test (20 sequences)	RT-DETR-L; YOLOv8n	ByteTrack	predeclared held-out
KITTI tracking training, also with halved scores	RT-DETR-L; YOLOv8n	ByteTrack (RT-DETR-L); OC-SORT (YOLOv8n)	development (baselines, ACT, and components)

$$\mathrm{HOTA} = \frac{1}{19} \sum_{\alpha} \sqrt{\mathrm{DetA}_{\alpha}\, \mathrm{AssA}_{\alpha}}, \tag{9}$$

where α runs over $0.05, 0.10, \ldots, 0.95$. DetA measures how well the objects are found, and AssA how well their identities are kept.

Benchmark protocols. The metrics are computed with the TrackEval library (commit 12c8791) and the official ground truth of each benchmark, following MOTChallenge practice [21]. On KITTI tracking, HOTA is the official value averaged over cars and pedestrians. On UAVDT, all vehicles form one class, and predictions inside the regions that the dataset marks as ignored are removed.

Statistical comparison. All comparisons are paired: each tracker runs twice on the same sequences and detections, once alone and once with SCL, and nothing else changes. To judge whether the difference between the two runs is reliable, paired bootstrap resampling is applied at the sequence level [24]. Resampling whole sequences keeps the dependence within each sequence and gives intervals for the differences. Each resample draws the evaluation sequences with replacement, 10,000 times with the fixed random seed 42. The same draw is used for both runs, the pooled metrics are recomputed, and their difference is recorded. The 95% interval is the 2.5–97.5 percentile range, and a difference is called significant when its interval does not include zero. An interval that includes zero does not show that there is no difference; it shows only that the tested sequences cannot separate the difference from zero. As a simple check that needs no statistics, we also count the sequences where HOTA rose, stayed the same ($|\Delta\mathrm{HOTA}| < 0.01$), or fell.

Units and direction. HOTA, MOTA, and IDF1 are given in percent, and differences between two runs in percentage points (points); relative changes are marked as such. IDS, FP, and FN are counts; runtime is given in milliseconds (ms) per frame. Higher values are better for HOTA, MOTA, and IDF1, and lower values are better for IDS, FP, FN, and runtime.

2.3 MOT Literature Survey

MOT has been widely studied, with research focusing on better detectors, better association, and better tracking under changing conditions. Three lines of this work relate directly to SCL. Tracking-by-detection methods provide the trackers that SCL serves. Adaptive threshold methods change thresholds inside a tracker and are the closest alternatives to SCL. Detector calibration and adaptive inference methods change the detector scores or how a model runs. The following subsections review these three lines in this order.

2.3.1 Tracking-by-Detection Methods in MOT

Tracking-by-detection is one of the most widely used approaches in MOT [3]. Objects are detected in each frame and then associated across frames to keep their identities. SORT predicts where each tracked box will be in the next frame with a Kalman filter, a standard motion predictor, and matches these predictions to the new detections

by their overlap, using the Hungarian method, which finds the best one-to-one matching [4]. ByteTrack added a second matching step that uses low-score detections to continue existing tracks [5], and many later trackers kept this two-stage design. Others, such as OC-SORT [6], use only one score threshold. All these trackers read detector scores through fixed thresholds that were tuned for one detector, usually YOLOX [18]. A recent survey groups the tracking-by-detection methods into joint detection and embedding, heuristic, motion-based, affinity-learning, and offline methods, and compares them across benchmarks [3].

Many recent trackers improve their association with parts that are trained or designed for a domain: small re-identification networks, which compare the appearance of objects [25], learnable graph matching [26, 27], result-guided feature extraction for faster tracking [28], motion models for low-frame-rate surface-vehicle videos [29], and image restoration by blind super-resolution before tracking in adverse weather [30]. These methods change or retrain the tracker. SCL keeps both the tracker and the detector unchanged and acts only on what passes between them.

Trackers and detectors used in this study. Seven published trackers serve as hosts, that is, as the trackers that SCL serves: ByteTrack [5], BoT-SORT [31], OC-SORT [6], SparseTrack [32], BoostTrack [33], PD-SORT [34], and Hybrid-SORT [35]. Each host uses the authors' code and published configuration. ByteTrack is used in two published settings: the official setting, with the thresholds that its authors used on MOT17, and the library setting, with the default thresholds of the widely used Ultralytics library, which also provides the BoT-SORT host. The detectors are YOLOX-X [18], YOLOv8n [19], and RT-DETR-L [20]. All hosts and detectors are used without modification.

2.3.2 Adaptive Threshold Selection in MOT

Several studies adapt the thresholds of a tracker to changing conditions. Adaptive ByteTrack sets the score threshold of ByteTrack from the detections of each frame [9]. AdapTrack adapts the thresholds of its matching step [10]. ACT sets the high and low thresholds of each frame from the first quartile of the scores and the number of candidates [11]. Other recent trackers use the score or the spread of each detection inside the tracker. ConfTrack weights the Kalman update, the Kalman noise, and the matching cost by the score of each box [36]. UTrack measures the spread of the boxes that non-maximum suppression (NMS), the detector step that removes overlapping boxes of the same object, merges, and uses it as the Kalman noise, so it needs the boxes before this step [37]. PlugTrack blends the Kalman filter with learned motion predictors through a small network trained with labels [38]. These methods are built into the tracker, use only the current frame or each box alone, or need extra trained parts or the inside of the detector.

SCL follows a different approach and differs in four ways. It uses past frames. It decides for the whole video whether to act at all, and otherwise keeps the tracker's own settings. It works through a host contract, four published settings of each tracker, on several published trackers without changing them. And it is tested as one frozen configuration under score changes and on external systems.

2.3.3 Detector Confidence Calibration and Adaptive Inference

Scores do not mean the same thing for every detector. Detectors of different families, such as YOLOX [18], YOLOv8 [19], YOLOv9 [39], DETR [40], and RT-DETR [20], give different score distributions. Their confidence is not calibrated across models and conditions, and calibration methods for object detection learn a mapping from labelled data [7]. A tracker tuned for one detector therefore cannot assume that another detector's scores mean the same thing, and labels to recalibrate them are rarely available after deployment. Image processing offers a tool that needs no labels: Otsu's method picks a threshold from a histogram so that the two resulting groups are as separate as possible [14]. SCL applies it to the logits (log-odds) of the detector scores of recent frames.

A wider group of methods adapts inference itself. Dynamic neural networks adapt their computation to each input [12], and Chameleon measures again, from time to time, which settings of a video analysis system work best [13]. These methods learn or measure their settings for each model.

Table 3 Comparison of SCL with recent adaptive methods for tracking (BT: ByteTrack; ReID: re-identification network; NMS: non-maximum suppression).

Method	Year	Adapted	Statistics	Extra trained part	Detector input	Where it works
Adapt. BT [9]	2023	high/low score split	current frame	none	boxes and scores	inside BT
ConfTrack [36]	2024	Kalman update, noise, and cost	score of each box	ReID	boxes and scores	new tracker
AdapTrack [10]	2024	matching thresholds	current frame	ReID	boxes and scores	new tracker
UTrack [37]	2025	Kalman noise	spread of boxes merged by NMS	none	boxes before NMS	added to existing trackers
ACT [11]	2026	high and low thresholds	current frame	none	boxes and scores	inside the tracker
PlugTrack [38]	2026	mix of Kalman and learned motion	track boxes	network trained with labels	boxes	added to two learned predictors
Küppers [7]	2020	detector scores	labelled data	calibration map	boxes and scores	detector output
Chameleon [13]	2018	pipeline configuration	profiling	none	whole pipeline	analytics pipeline
AC-MOT (ours, submitted)	2026	detector operating point	image and track cues	none	sets its settings	before the detector
SCL (this paper)	–	candidates, scores, thresholds	past frames, no labels	none	boxes and scores	between detector and seven unchanged trackers

Our earlier work, AC-MOT (a manuscript submitted for publication), is a controller that needs no training and sets the detector operating point (score threshold, NMS threshold, and input resolution) while running, from image and track cues. Such a controller is tied to the detector it was calibrated with, because a change in the detector's score scale does not change the image. SCL reads the scores instead and acts on the tracker's input and thresholds.

In summary, existing work either adapts thresholds or uses the score of each box inside one tracker, calibrates scores with labelled data, or learns its settings for one model. A layer that needs no training, adjusts itself from the detector's own scores, works with many unchanged trackers, and keeps a well-matched tracker at its own accuracy is missing. SCL is designed to fill this gap. Table 3 summarizes the main differences between SCL and the closest adaptive methods.

3 Proposed SCL Framework

SCL is a small, rule-based layer that runs once per frame between the detector and the tracker. In every frame it does four things in order. First, it looks at the detector scores of the last few frames and puts the candidates into three groups: background, uncertain, and confident (Sect. 3.2). Second, it uses these groups to decide whether the score distribution is clean or noisy (Sect. 3.3). Third, it acts on this decision: on clean scores it keeps the tracker's own settings, and on noisy scores it chooses which candidates reach the tracker, assigns them new scores, and sets the tracker's thresholds. Fourth, in both cases it can pass a weak candidate that clearly continues an existing track (Sect. 3.4). The tracker then runs as usual on what it receives. The detector and the tracker are never changed. Section 3.6 explains how each value of the layer is chosen, and Table 4 lists the symbols.

3.1 Overview and inputs

Figure 1 shows where SCL sits in the pipeline: it receives the candidates of the detector, decides what the tracker will see, and passes its decision to the tracker, which then runs as usual. In each frame t, the detector gives a list of candidates $D_t = \{(\mathbf{x}_i, s_i, c_i)\}$: a box $\mathbf{x}_i$, a score $s_i \in (0, 1)$, and a class label c_i for each one, down to a lowest score called the emission floor. The layer does not use the score itself but its logit (log-odds), which

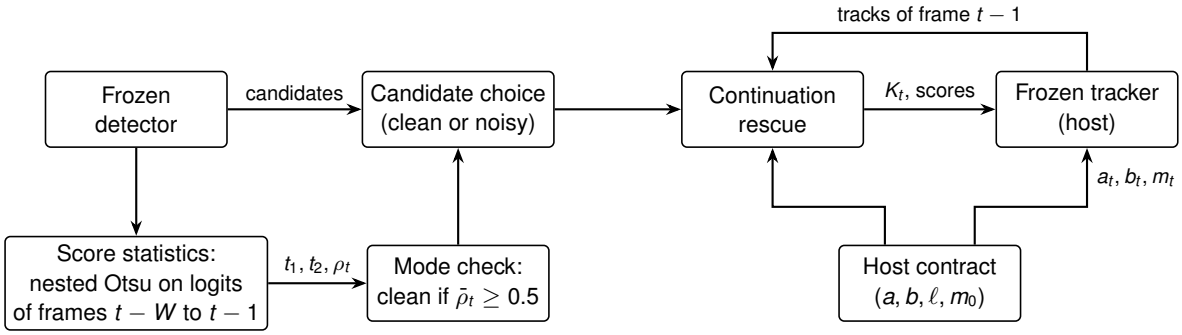


Fig. 1 SCL between a frozen detector and a frozen tracker (the host). Boxes are processing steps and arrows show what each step passes on. The score statistics use earlier frames only. The host contract holds the tracker's published settings. The tracker receives the chosen candidates K_t, their scores, and its thresholds a_t, b_t and tolerance m_t for the frame

spreads out scores close to 0 and 1:

$$z_i = \log \frac{s_i}{1 - s_i}. \tag{10}$$

For example, a score of 0.5 gives $z_i = 0$, a score of 0.9 gives $z_i \approx 2.2$, and a score of 0.1 gives $z_i \approx -2.2$. The logistic function $\sigma(z) = 1/(1 + e^{-z})$ turns a logit back into a score. Table 4 lists all symbols of the method.

As defined in Sect. 1, the tracker that the layer serves is called the host. It gives the layer four numbers about itself, the host contract $h = (a, b, \ell, m_0)$. These are simply the settings that its authors published: a is the score a detection needs to be matched to a track in the first step, b is the score it needs to start a new track, ℓ is the lowest score the tracker uses at all, and m_0 is its matching tolerance (one minus the smallest overlap that still counts as a match). Overlap is measured by the intersection over union (IoU) of two boxes: the area they share divided by the area they cover together, from 0 (no overlap) to 1 (the same box). A single-stage tracker uses one threshold, so $\ell = \min(a, b)$. A two-stage tracker also uses weaker detections in a second step, so $\ell < \min(a, b)$.

In every frame the layer returns five things: the candidates K_t that the tracker will see, their scores $\tilde{s}$, and the thresholds (a_t, b_t) and tolerance m_t that the tracker will use in this frame:

$$(K_t, \tilde{s}, a_t, b_t, m_t) = \mathcal{L}\left(Z_{<t}, D_t, \mathcal{T}_{t-1}, g_t; h\right). \tag{11}$$

Here $Z_{<t}$ are the logits of the earlier frames, $\mathcal{T}_{t-1}$ are the tracks of the previous frame, and g_t is an optional measure of camera motion in frame t. In words, Eq. (11) says that the output for frame t depends only on the logits of earlier frames, the candidates of frame t, the tracks of frame $t-1$, the camera motion, and the tracker's contract. The decision for frame t uses only past frames; frame t is added to the statistics after the decision, and no future frame is ever used.

Four goals guided the design. The layer needs no training and no labels. One frozen configuration must work for every tracker, detector, and

Table 4 Main symbols of the method.

Symbol	Meaning
t	frame index
D_t	candidates of frame t given by the detector
$\mathbf{x}_i, s_i, c_i$	box, score, and class label of candidate i
z_i	logit (log-odds) of the score s_i, Eq. (10)
$\sigma(\cdot)$	logistic function, turns a logit back into a score
$h = (a, b, \ell, m_0)$	host contract: the tracker's published settings
a, b	score needed to match a track, and to start a new track
ℓ	lowest score the tracker uses
m_0	matching tolerance: one minus the smallest IoU that counts as a match
W	number of past frames used for the statistics (10)
t_1, t_2	first and second Otsu split on the logits
Z_t^W	logits of the last W frames, $t-W$ to $t-1$
ρ_t, $\bar{\rho}_t$	share of confident logits in Z_t^W, and the median of all ρ values so far
u_i	rank of candidate i among recent scores, from 0 to 1
K_t, $\tilde{s}$	candidates passed to the tracker, and their scores
a_t, b_t, m_t	thresholds and tolerance used by the tracker in frame t
$\mathcal{T}_{t-1}$	tracks of the previous frame
g_t, r_t	camera motion in frame t, and its ratio to the usual motion
IoU	intersection over union: overlap of two boxes, from 0 to 1

dataset. When the tracker already fits the detector, its input must stay the same. And the layer must take little time compared with the detector.

3.2 Self-calibration from detector scores

A detector that suits the scene gives many low scores to background and a separate group of high scores to objects. The layer checks, without labels, how clearly this happens. It collects the logits of the candidates of the last $W = 10$ frames that remain after duplicate removal (Sect. 3.3) and splits them twice with Otsu's method [14], which finds the threshold that best separates a histogram into two groups. The first split, t_1, separates background (low logits) from foreground (high logits). The second split, t_2, is found in the same way on the foreground logits only, and divides them into uncertain and confident candidates. We call these two splits nested Otsu splits. The share of confident candidates in the foreground,

$$\rho_t = \frac{\left|\{z \in Z_t^W : z \geq t_2\}\right|}{\left|\{z \in Z_t^W : z \geq t_1\}\right|}, \tag{12}$$

where Z_t^W are the logits of the last W frames, $t - W$ to $t - 1$, the same logits that give t_1 and t_2. It describes the shape of the score distribution: how much of the foreground is clearly confident. It is a heuristic indicator, not a measure of detection accuracy; a detector that gave high scores to many false boxes would also give a high ρ_t. In words, ρ_t is the number of confident candidates divided by the number of foreground candidates in these frames. It is computed before frame t is processed; the candidates of frame t enter the statistics only after the decision for frame t. For example, if 40 candidates are in the foreground and 30 of them are confident, then $\rho_t = 0.75$. A high ρ_t means that most foreground candidates are confident; a low ρ_t means that many of them are uncertain. Because the splits are made on logits, they move with the scores when the whole logit scale is stretched or shifted, for example by a change of temperature, and each group keeps the same candidates. Figure 2 shows the logits and the three groups for a clean stream and for a noisy stream.

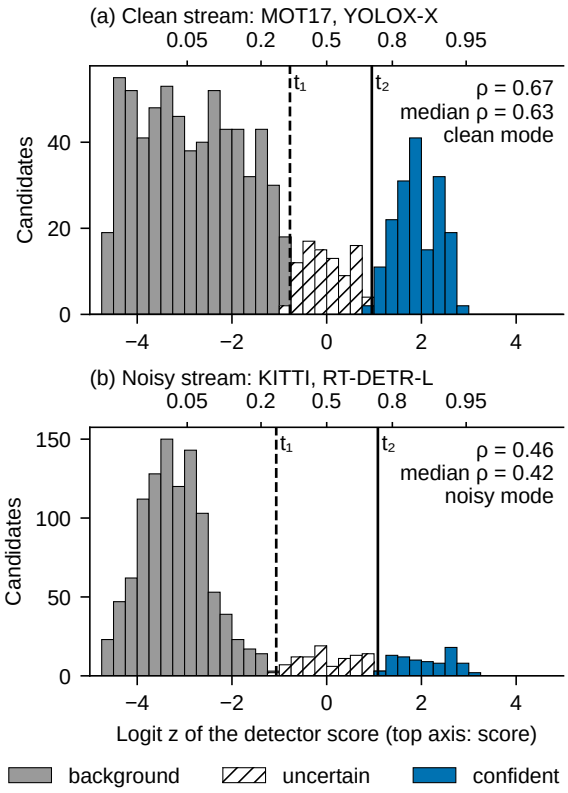


Fig. 2 What the layer sees: the logits of the candidates of the last 10 frames, just before its decision for frame 101 of a sequence, with the two Otsu splits t_1 (dashed line) and t_2 (solid line). Plain light bars: background; hatched bars: uncertain; plain dark bars: confident. The top axis gives the detector score that matches each logit. (a) MOT17-02 with the published YOLOX-X detections: most foreground candidates are confident, and the stream is clean, so the tracker keeps its settings. (b) KITTI sequence 0001 with RT-DETR-L: many candidates are uncertain, and the stream is noisy, so the layer chooses the candidates and sets the thresholds. All values are those of the frozen layer in the runs of Sect. 5

These groups only make sense if there is a real background group. Some detectors give very few low scores, for example because they drop every box below a high emission floor; then the first split falls inside the objects, and the lower group holds weaker objects rather than clutter. The layer detects this case with a simple heuristic: when a detector emits its low scores, clutter usually gives more candidates than the objects do, so a lower group that is smaller than the upper one, that is, fewer candidates below t_1 than above it, is taken as a sign that no background group was emitted. The frame then gives no evidence

against the host's settings and is counted as clean, $\rho_t = 1$. This background check covers, for example, the first frames of a sequence with only about ten candidates per frame, where both splits would otherwise fall among the objects ($t_1 \approx 0.6$ and $t_2 \approx 0.93$ in probability terms).

3.3 Clean and noisy modes

The mode decides how much the layer may change. Clean and noisy describe the score distribution of the stream, not the accuracy of its detections. In a clean stream, most foreground candidates fall in the confident group; the layer then assumes that the tracker's own thresholds fit the scores and keeps them. In a noisy stream, many foreground candidates are uncertain, so the layer takes over the choice of candidates and thresholds.

Choosing the mode. The stream is clean at frame t if the median $\bar{\rho}_t$ of all ρ values so far (ignoring cold frames) is at least one half, and noisy otherwise. The level of one half is a majority rule, not a fitted value: the stream is clean when confident candidates are the majority of the foreground in at least half of the frames. Before the first valid split, for example in the first frame, the mode is called cold. The decision is made online, from the frames seen so far. Because the median covers the whole stream so far, the mode is stable: a short drop in confidence, for example during fast camera motion, does not change it. The price is a slower response to a lasting change: if the scores change for good in the middle of a long stream, the mode follows only when the frames after the change make up about half of the frames seen so far. Stability was preferred because, during development, a mode that changed in the middle of a stream lowered HOTA (Sect. 4.2).

Cold and clean modes. The tracker keeps its own thresholds. In the clean mode there is one exception: if a threshold is so high that it would reject the confident group, it is lowered to t_2:

$$a_t = \min(a, \sigma(t_2)), \qquad b_t = \min(b, \sigma(t_2)), \quad (13)$$

where σ is the logistic function, which turns a logit back into a score. In words, each threshold stays as the tracker set it, unless it is higher than the score $\sigma(t_2)$ at which the confident group begins; then it is lowered to that score. Scores are passed unchanged, and only cross-class duplicates (one box given two class labels) are removed. When the confident group already lies above the tracker's thresholds, the tracker therefore receives exactly its normal input.

Example. When the detector scores are halved, no score is above 0.5. A tracker with $a = 0.6$ and $b = 0.7$, such as ByteTrack in its official setting (Table 5), then receives no detection that can start a track, so it never creates a track. The Otsu splits are computed from the halved scores themselves, so they move with them. In the clean mode, Eq. (13) then lowers the tracker's thresholds to $\sigma(t_2)$, and the confident candidates reach the tracker again. In the noisy mode, the new scores below place them at or above the thresholds of 0.5. Section 5.3 shows the measured effect.

Noisy mode. Here the layer decides which candidates the tracker may use, and for what. Confident candidates ($z \geq t_2$) may start and continue tracks. Uncertain candidates ($t_1 \leq z < t_2$) may only continue existing tracks. Background candidates ($z < t_1$) are dropped. Each score is replaced by a new one based on its rank u_i among recent scores. These are kept in a small memory, the remap memory: the scores of 20 past frames, one frame taken every 10 frames. The rank is the position of the score in this memory (from 0 for the lowest to 1 for the highest),

$$\tilde{s}_i = \begin{cases} 0.5 + 0.5\,u_i, & z_i \geq t_2, \\ 0.1 + 0.4\,u_i, & t_1 \leq z_i < t_2, \end{cases} \quad (14)$$

and the tracker's matching and birth thresholds are both set to 0.5, $a_t = b_t = 0.5$. In words, the lowest-ranked confident candidate gets 0.5 and the highest-ranked one gets close to 1, while an uncertain candidate gets a score between 0.1 and 0.5 in the same order. Confident candidates thus get scores of 0.5 and above, and uncertain ones get scores below 0.5, so they can only reach a tracker's low-score step. Duplicates are removed with the help of the tracks: a weaker candidate that overlaps a stronger one with an intersection over union (IoU) above one half is removed, unless a different track of the previous frame claims it. This keeps people who stand close together in a crowd.

How the layer helps identities. An identity switch happens when a track takes the box of another object, or when a lost object is found

again and starts a new track with a new ID. The layer is designed to reduce both cases. In the noisy mode it keeps background candidates away from the tracker, so fewer false boxes can be matched to a track. The continuation rescue (Sect. 3.4) keeps a track alive when its object gets low scores for a few frames, so the object keeps its ID. The duplicate rule uses the tracks of the previous frame, so a box that belongs to a different track is not removed and its object is not lost.

3.4 Track continuation and camera motion

A tracker ignores every candidate below its lowest threshold, so a track can be lost when its object gets low scores for a few frames. This matters most for single-stage trackers, which have no low-score step. The rescue gives such a candidate a second chance, but only when it clearly continues an existing track. Three conditions must hold: the candidate is in the foreground ($z \geq t_1$, or any candidate for a single-stage tracker when there is no background group); it overlaps a track of the previous frame with an IoU of at least one half; and no other usable candidate covers that track. Also, the tracker must be unable to see the candidate with the thresholds of this frame. The candidate is then passed with its score raised just above the tracker's lowest threshold. For a two-stage tracker, this only applies to candidates below its low threshold ℓ. For a single-stage tracker, it applies to candidates below the lowest threshold used in this frame, so the rescue gives it the continuation step it lacks. A rescued candidate can still be a false box that happens to overlap a track; the conditions above limit this case but do not exclude it, so a wrong box can extend a track. The identity switches of the full layer, with the rescue, are reported for every cell in Sect. 5.

When the camera moves fast, a box moves more between two frames, and a fixed overlap requirement can break a correct match. In noisy frames, and only when a motion measure is available, the matching tolerance therefore grows with the current camera motion; r_t is the current motion divided by its median over the last 100 frames (used after a warm-up of 5 frames). The motion is the shift between two consecutive grayscale frames at quarter resolution, found by phase correlation, a standard method that measures how far one image is shifted against another [41], and divided by the image diagonal. The tolerance is

$$m_t = \min\left(0.95,\ 1 - \frac{1 - m_0}{\max(1, r_t)}\right). \tag{15}$$

In words, when the camera moves no faster than usual ($r_t \leq 1$), the tolerance stays at m_0. When the motion is twice its usual value ($r_t = 2$), the gap $1 - m_0$ is halved: with $m_0 = 0.8$, the tolerance becomes 0.9, so a match needs an IoU of only 0.1 instead of 0.2. The tolerance never exceeds 0.95.

3.5 Host contract and adapters

Table 5 lists the contracts used in this paper. Each value is taken from the tracker's published configuration; none is fitted. A small piece of code for each tracker family, called an adapter, passes the layer's decision to the tracker: the chosen candidates and their scores replace the detector output of the frame, and the new thresholds and tolerance replace the tracker's own for that frame. The adapter does not change the tracker's code: before each update it writes these values into the tracker's existing parameters, for example the detection threshold and the matching overlap of OC-SORT, and then calls the tracker's own update. The matching steps, the motion model, and the track management of the tracker run unchanged. Any step that the authors apply after tracking, such as filling short gaps in tracks (interpolation) or linking tracks after the video ends, is kept. In this sense the host stays frozen: its code, its learned parts, and its stored configuration do not change, and only the values of its contract thresholds may differ from frame to frame. The rules and their constants are the same for every tracker. A tracker enters only through its four contract values, and the adapter only passes on the decision; it never makes one.

3.6 Selection of Parameter Values of SCL

All values of Table 6 are fixed before the external evaluation and are not changed after it, and the same values are used for every tracker, detector, and dataset. The thresholds t_1 and t_2 are not constants: they are computed again in every frame.

Table 5 Host contracts: the published settings of each tracker, as used by the layer.

Tracker (setting)	Type	assoc a	birth b	low ℓ	match m_0	Decision mapped to
ByteTrack (official MOT17)	two-stage	0.6	0.7	0.1	0.8	thresholds, candidates, scores
ByteTrack (library default)	two-stage	0.25	0.25	0.1	0.8	same
BoT-SORT (library default)	two-stage	0.25	0.25	0.1	0.8	same
SparseTrack	two-stage	0.6	0.7	0.1	0.85	same
BoostTrack	two-stage	0.6	0.6	0.1	0.7	same
Hybrid-SORT	two-stage	0.6	0.6	0.1	0.75	same
OC-SORT	single-stage	0.6	0.6	0.6	0.7	same
PD-SORT	single-stage	0.6	0.6	0.6	0.7	same

The last column of Table 6 gives the source of each value. There are three sources. All values are engineering choices with a stated reason, and none of them was chosen by searching over a range of values. The paper therefore does not claim that they are the best values, only that one fixed set worked across the tested systems.

Standard values of MOT evaluation and of the trackers. The overlap of one half for duplicates and rescue is the standard matching threshold of MOT evaluation, an IoU of 0.5. The score ranges of Eq. (14) put the two groups on the common thresholds of 0.5 (matching and birth) and 0.1 (low score).

Definitions. The mode rule calls a stream clean when, in at least half of the frames, at least half of the foreground candidates are confident (a majority rule), and the background check marks a window without a background group (Sect. 3.2). The rescued score is placed just above the lowest threshold of the frame, and the cap of 0.95 keeps a minimum overlap of 0.05 for every match.

Design choices. The window, the remap memory, and the motion history were set at the start of development and kept fixed in all development stages (Sect. 4); they were not searched.

3.7 Per-frame steps and cost

Algorithm 1 gives the steps for one frame. The work per frame is small: the layer sorts the logits of at most W frames for the two Otsu splits, computes overlaps between candidates and with the tracks of the previous frame, and, when motion is used, computes one phase correlation on a quarter-resolution image. With n candidates per frame and k tracks, the cost is $O(Wn \log(Wn) + n^2 + nk)$ plus a fixed image cost. The three terms come from sorting the logits of the window, comparing every pair of candidates, and comparing every candidate with every track. The layer stores only the logits of the last W frames, the per-frame ρ values, the remap memory, and the last 100 motion values. Table 7 and Fig. 3 give the measured time per frame of a full pipeline on an ordinary processor, with and without SCL.

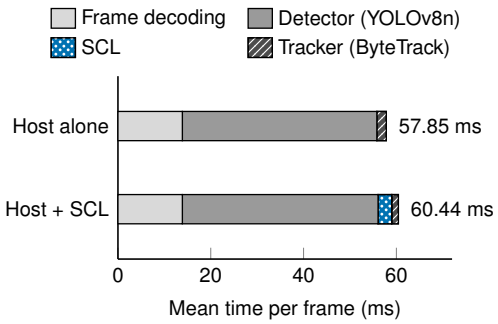


Fig. 3 Mean time per frame of each stage for the pipeline with the tracker alone and with SCL (4-vCPU Intel Xeon, no GPU). Light and medium plain segments: frame decoding and detector; dotted segment: SCL; striped segment: tracker. The labels give the measured end-to-end time

4 Development of the Final SCL

The layer of Sect. 3 is the result of three development stages. The starting point (Sect. 4.1) asked whether an earlier design that always intervened, AOB, can be placed in front of strong published trackers; the design steps (Sect. 4.2) removed the causes of its losses one at a time, and each step was compared with the step it was built on; and the last stage (Sect. 4.3) froze the final configuration before any external test. All development runs used the development cells of Sect. 2.1, and no value was changed after the freeze.

Table 6 Values of SCL and the source of each value.

Value	Setting	Source
Statistics window W	10 frames	design choice, fixed in all development stages
Mode rule	clean if median $\rho \geq 0.5$ over non-cold frames so far	definition (majority rule): at least half of the foreground is confident
Background check	$\rho = 1$ if fewer candidates below t_1 than above	heuristic: no background group in the window
Noisy score ranges	primary $[0.5, 1)$, extension $[0.1, 0.5)$	common thresholds 0.5 and 0.1
Noisy association and birth thresholds	0.5	common threshold 0.5
Remap memory	20 frames, one every 10 frames	design choice, fixed in all development stages
Duplicate and rescue IoU	0.5	standard matching IoU of MOT evaluation
Rescued score	lowest used threshold + 0.001	just above the lowest threshold of the frame
Motion cue	phase correlation, quarter resolution	standard method [41]
Motion history, warm-up	100, 5 frames	design choice, fixed in all development stages
Tolerance cap	0.95	keeps a minimum overlap of 0.05

Table 7 Baseline comparison of the per-frame time on a 4-vCPU Intel Xeon (2.10 GHz, no GPU) with a live YOLOv8n detector (736 pixels) and ByteTrack on three KITTI sequences (2,279 timed frames per arm). Host alone: the same pipeline without SCL. Mean / 95th percentile in milliseconds.

Stage	Host alone	Host + SCL
Frame decode	13.88 / 16.21	13.88 / 16.51
Detector (YOLOv8n, 736 px)	41.91 / 58.41	42.17 / 59.43
SCL	–	2.95 / 4.39
Tracker (ByteTrack)	2.04 / 3.57	1.43 / 2.64
End to end	57.85 / 76.35	60.44 / 80.78

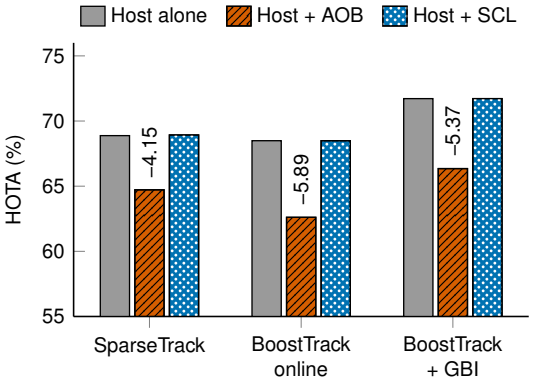


Fig. 4 HOTA on MOT17 val-half of strong published trackers (SparseTrack and BoostTrack in two settings). Plain bars: the tracker alone; striped bars: with AOB; dotted bars: with SCL. The vertical labels give the change caused by AOB in HOTA points

4.1 Starting point: always-on banding

SCL was developed from an earlier design of the same research program, called always-on banding (AOB). AOB computed the same nested Otsu groups, or score bands, but treated every stream as noisy, so it applied the bands in every frame; this is where its name comes from. In every frame it also removed every weaker candidate that overlapped a stronger one with IoU above one half, and gave all candidates new scores by rank on fixed ranges. It passed no candidate in the first frame. AOB was designed for noisy detector streams, and it is the baseline of this section.

Table 8 compares AOB and SCL with the same strong published trackers on MOT17 [15]. AOB lowered the HOTA of SparseTrack by 4.15 points and that of BoostTrack by 5.89 points online and 5.37 points with BoostTrack's post-processing that fills short gaps in tracks (linear and gradient boosting interpolation, GBI), all with intervals below zero. Two causes were found. First, removing overlapping boxes as duplicates deleted people hidden behind others in crowds. Second, the self-calibrated thresholds were stricter than the trackers' own: for SparseTrack and online BoostTrack, FP fell by 1,630 and 1,351, but FN rose by 4,955 and 6,167. On the same trackers, SCL changed HOTA by +0.06 for SparseTrack and left both BoostTrack versions identical to the host. Figure 4 shows the same comparison.

What this stage showed. A layer that acts in every frame harms trackers that already fit the detector: strong published trackers lost between 4.15 and 5.89 HOTA points in all three settings. Such a layer cannot be placed safely in front of an unknown tracker. The next steps therefore let the layer decide from the scores whether to act at all.

4.2 Design steps

Each step from AOB to SCL was made to remove one cause found before it, and it was compared with the step before it on development data

Algorithm 1 SCL for frame t. The window holds the logits of frames $t-W$ to $t-1$, so the decision for frame t uses no future frame.

Input: candidates D_t, tracks $\mathcal{T}_{t-1}$, motion cue g_t, contract (a, b, ℓ, m_0), constants of Table 6
Output: candidates K_t, scores $\tilde{s}$, thresholds a_t, b_t, tolerance m_t

$z_i \leftarrow$ logit of s_i for all i ▷ Eq. (10)
if no valid split exists **then** ▷ e.g., first frame
  mode $\leftarrow$ cold; $(a_t, b_t) \leftarrow (a, b)$
else
  $(t_1, t_2) \leftarrow$ nested Otsu on the window ▷ Sect. 3.2
  $\rho_t \leftarrow$ Eq. (12) with the background check
  mode $\leftarrow$ clean if median(ρ) ≥ 0.5, else noisy
end if
if mode is cold or clean **then** ▷ Sect. 3.3
  remove cross-class duplicates; keep raw scores
  if clean **then** set (a_t, b_t) by Eq. (13)
  end if
else ▷ noisy mode
  remove duplicates with track context
  drop $z < t_1$; remap scores by Eq. (14)
  $a_t \leftarrow 0.5$; $b_t \leftarrow 0.5$
end if
rescue continuations (not cold) ▷ Sect. 3.4
$m_t \leftarrow$ Eq. (15) if noisy, else m_0
pass K_t, $\tilde{s}$, a_t, b_t, m_t to the tracker
add the logits left after duplicate removal to the window
add g_t to the motion history
store the tracks of frame t after the tracker update

(Table 9). The steps, in the order in which they were made, were the following.

- *Starting point.* Running AOB inside the new framework showed the same problem: it reduced ByteTrack at floor 0.1 from 67.70 to 56.41 HOTA and OC-SORT from 66.44 to 45.72.
- *Mode gate.* The two causes found in Sect. 4.1 were thresholds stricter than the trackers' own and the removal of hidden people as duplicates. The mode gate adds the mode check, the clean mode that keeps the tracker's settings, and the crowd-safe duplicate rule. It restored SparseTrack from 64.72 to 68.93 and BoostTrack from 62.61 to 68.37.

Table 8 Baseline comparison of the development stage on MOT17 val-half (published YOLOX-X detections). Host: the tracker alone, without any layer (our run of the published tracker). Δ: tracker with the earlier design AOB or with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42). Bold intervals exclude zero. IDS: identity switches, a count (lower is better).

Metric	Host	Δ AOB	Δ SCL
SparseTrack [32]			
HOTA	68.88	**−4.15**	+0.06
		[−5.54, −1.66]	[−0.01, +0.25]
MOTA	77.85	**−6.14**	+0.08
		[−8.10, −2.74]	[−0.03, +0.30]
IDF1	81.97	**−4.49**	+0.16
		[−6.19, −1.96]	[−0.02, +0.70]
IDS	124	−16	+7
		[−47, +12]	[0, +19]
BoostTrack, online [33]			
HOTA	68.49	**−5.89**	0 (identical)
		[−7.76, −2.87]	
MOTA	75.50	**−8.87**	0 (identical)
		[−12.37, −7.14]	
IDF1	81.41	**−6.22**	0 (identical)
		[−9.33, −2.85]	
IDS	113	**−38**	0 (identical)
		[−81, −1]	
BoostTrack, with GBI [33]			
HOTA	71.72	**−5.37**	0 (identical)
		[−7.30, −2.75]	
MOTA	81.03	**−8.73**	0 (identical)
		[−12.59, −6.50]	
IDF1	84.16	**−5.65**	0 (identical)
		[−8.96, −2.77]	
IDS	97	−27	0 (identical)
		[−73, +11]	

- *Whole-stream mode.* The mode still changed in the middle of some streams. Choosing it from the whole stream removed these changes and raised ByteTrack from 67.561 to 67.702 and OC-SORT from 65.697 to 66.071.
- *Continuation rescue.* Single-stage trackers still lost tracks whose objects got low scores for a few frames. The rescue added 0.2 to 0.4 points for OC-SORT.
- *Background check.* Streams with few low-score candidates need a separate rule (Sect. 3.2). The background check completed SCL: it returned ByteTrack and BoostTrack to output identical to the host and raised OC-SORT to 67.04 and 66.91 at the two floors.

Four other variants were also tested at these steps and were not kept (Table 9).

What this stage showed. After these steps, the layer kept the strong two-stage trackers at

their own accuracy and improved the single-stage OC-SORT. This is the behavior that the frozen configuration was then tested for on new trackers, detectors, and data (Sect. 5).

4.3 Freezing and testing

The final configuration was frozen before any external test. It was saved as a fixed, named version of the code (a version-control tag) and protected by a lock file that stores a fingerprint (cryptographic hash) of the layer, its configuration, and the evaluation scripts, so that any later change would be detected. Every later run checked the lock before running. Sixty-one automated tests check that the layer uses no future data, that its state resets between sequences, that clean streams pass through unchanged, that the output does not depend on detector, tracker, or sequence names, that stretching or shifting all logits leaves the groups unchanged where designed, and that the lock is intact.

5 Results

The experiments test whether adding the frozen SCL between an unchanged detector and an unchanged tracker keeps or improves tracking, on MOT17 and KITTI, on two external trackers, and, without retuning, on UAVDT. Three outcomes are expected from the design. A tracker that already fits its detector should keep its accuracy. A tracker that does not fit, because it has no continuation step or because the detector or its score scale changed, should gain. Both should hold on trackers and data that were not seen during design.

All experiments use the hosts and detectors of Sect. 2.3.1, each with the authors' code and published configuration, and the contract of each host is given in Table 5. For each host, two systems are compared. The baseline is the host alone (the host-only baseline), called Host in the tables: the published tracker without SCL, run by us with its published configuration and the same detections. The second system adds the frozen SCL of Sect. 3 and changes nothing else. Both runs use the same environment. Where the authors report their own numbers, these are listed next to our reproduction of the host. Uncertainty is estimated with the paired bootstrap of Sect. 2.2. Figure 5 shows

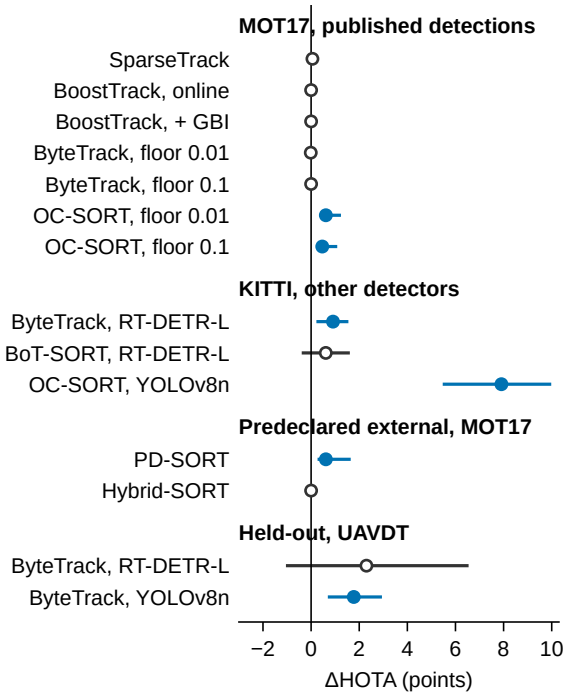


Fig. 5 Difference in HOTA, host with SCL minus host alone, in points, for every tested cell. The circle is the difference and the horizontal line its 95% paired-bootstrap interval. Filled circles: interval above zero; open circles: interval contains zero, or identical output

the HOTA difference between each host with SCL and the host alone for every tested cell, and the next subsections give the results of each part of the evaluation.

5.1 MOT17 evaluation across trackers

The MOT17 evaluation tests whether SCL leaves well-matched trackers at their own accuracy and helps a tracker that lacks a continuation step, with the published YOLOX-X detections. Table 10 compares the development trackers with their hosts on MOT17 val-half. The two-stage trackers with well-matched detections stayed at the accuracy of their hosts. BoostTrack, online and with GBI, and ByteTrack at floor 0.1 gave exactly the same output as without the layer, and ByteTrack at floor 0.01 and SparseTrack stayed within 0.06 HOTA points of their hosts (−0.014 and +0.055). This is the self-limiting behavior of the clean mode

Table 9 Design steps from AOB to SCL on development data (HOTA, %), and versions that were tested and rejected. The floor of ByteTrack and OC-SORT is given in parentheses.

Step	Change	Effect
AOB emulated	always-noisy bands, overlap deduplication, rank remap	ByteTrack (0.1) 56.41 vs host 67.70; OC-SORT (0.1) 45.72 vs 66.44
+ mode gate	mode statistic, host-anchored clean mode, crowd-safe duplicates	SparseTrack 64.72 to 68.93 (host 68.88); BoostTrack 62.61 to 68.37 (host 68.49)
+ whole-stream mode	mode from all past frames	ByteTrack (0.1) 67.561 to 67.702; OC-SORT (0.1) 65.697 to 66.071
+ continuation rescue	foreground track-consistent rescue	OC-SORT (0.01) 66.405 to 66.619; OC-SORT (0.1) 66.071 to 66.469
+ background check = SCL	no background mode gives no evidence against the host	ByteTrack/BoostTrack (0.1) identical to host; OC-SORT 67.041 (0.01), 66.907 (0.1)
Rejected version		*Evidence*
no candidates at frame 1		−0.3 to −0.5 HOTA on every MOT17 host
rescue below a two-stage host's low stage		FP +700 to +1000, −0.4 to −0.5 HOTA
rank-remapped scores in clean frames		ByteTrack −0.2 / −1.6 HOTA
host-clipped noisy primary band		KITTI YOLOv8n ByteTrack +1.4 HOTA but uncontrolled false tracks on a noisy aerial stream (26.9 vs 14.9 boxes/frame)

(Sect. 3.3), which the earlier design did not have (Table 8).

The single-stage OC-SORT improved at both emission floors. Its HOTA rose by 0.61 points ($[+0.36, +1.24]$) at floor 0.01 and by 0.46 points ($[+0.27, +1.09]$) at floor 0.1, and its MOTA by 1.27 and 1.26 points. Its identity switches also fell, by 8 at floor 0.01 ($[-15, -2]$) and by 14 at floor 0.1. HOTA was higher on 7 of 7 sequences in both settings (Fig. 6). The gain comes from the continuation rescue. OC-SORT drops every detection below its single threshold, and the layer passes it the foreground candidates that continue a track that would otherwise stay unmatched.

5.2 KITTI evaluation with other detectors

The KITTI evaluation tests what happens when the detector is replaced while each tracker keeps the thresholds that were set for another detector. Table 11 reports the results with the two other detectors. With the noisy RT-DETR-L detector, the two-stage trackers gained in MOTA and IDF1. ByteTrack gained 5.98 MOTA points ($[+1.16, +9.94]$), 3.87 IDF1 points, and 0.91 HOTA points ($[+0.22, +1.55]$). BoT-SORT gained 7.86 MOTA points ($[+2.21, +13.44]$) and 3.10 IDF1 points. Both also had fewer identity switches: 297 fewer for ByteTrack and 226 fewer for BoT-SORT. With YOLOv8n, the single-stage OC-SORT gained 7.91 HOTA points ($[+5.47, +9.98]$), 9.40 MOTA points, and 9.60 IDF1 points. It had 79 more identity switches, because it now followed many more objects: its missed objects (FN) fell from 21,886 to 16,576, and its IDF1, which measures how well identities are kept, rose. Figure 7 shows one frame of this cell: OC-SORT alone tracked 6 of the 10 labelled objects, and with SCL it tracked 9, including parked cars further down the street.

5.3 Evaluation under shifted detector scores

The shifted scores test the case for which SCL was built: the detector keeps its boxes, but its score scale changes. Table 12 and Fig. 8 show the trackers under the four score transforms of Sect. 2.1. Halving the scores reduced ByteTrack (official setting) and OC-SORT to 0 HOTA, because no detection reached their thresholds. With the layer they reached 66.17 and 66.72. Cubing reduced them to 60.92 and 58.34, and the layer restored them to 65.81 and 66.60. BoostTrack rose from 61.33 to 64.84 under cubing. With temperature $T = 2$, the layer raised ByteTrack and OC-SORT by 1.11 and 1.24 points. The library setting of ByteTrack, whose low thresholds already accept the changed scores, stayed within 0.14 points in all four transforms. Over the 14 conditions, the layer raised HOTA significantly in 7 and lowered it significantly in none. When a transform made a host lose most of its detections, the host also had few identity switches, because it followed few objects; under halved scores it followed none, so it had 0.

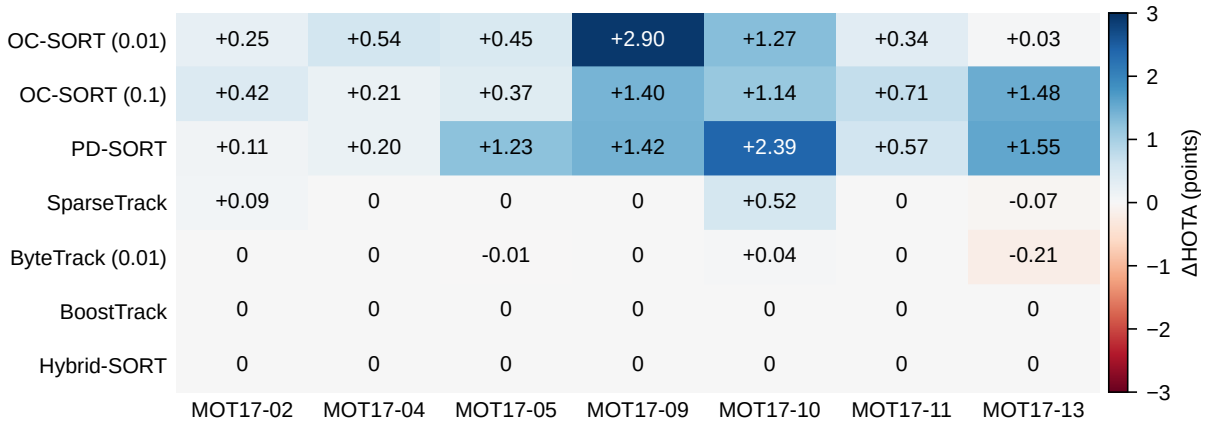


Fig. 6 HOTA difference, host with SCL minus host alone, in points, for each MOT17 val-half sequence (columns) and each development or predeclared tracker (rows). Each cell prints its value; the shade grows with the size of the difference, and 0 means identical output

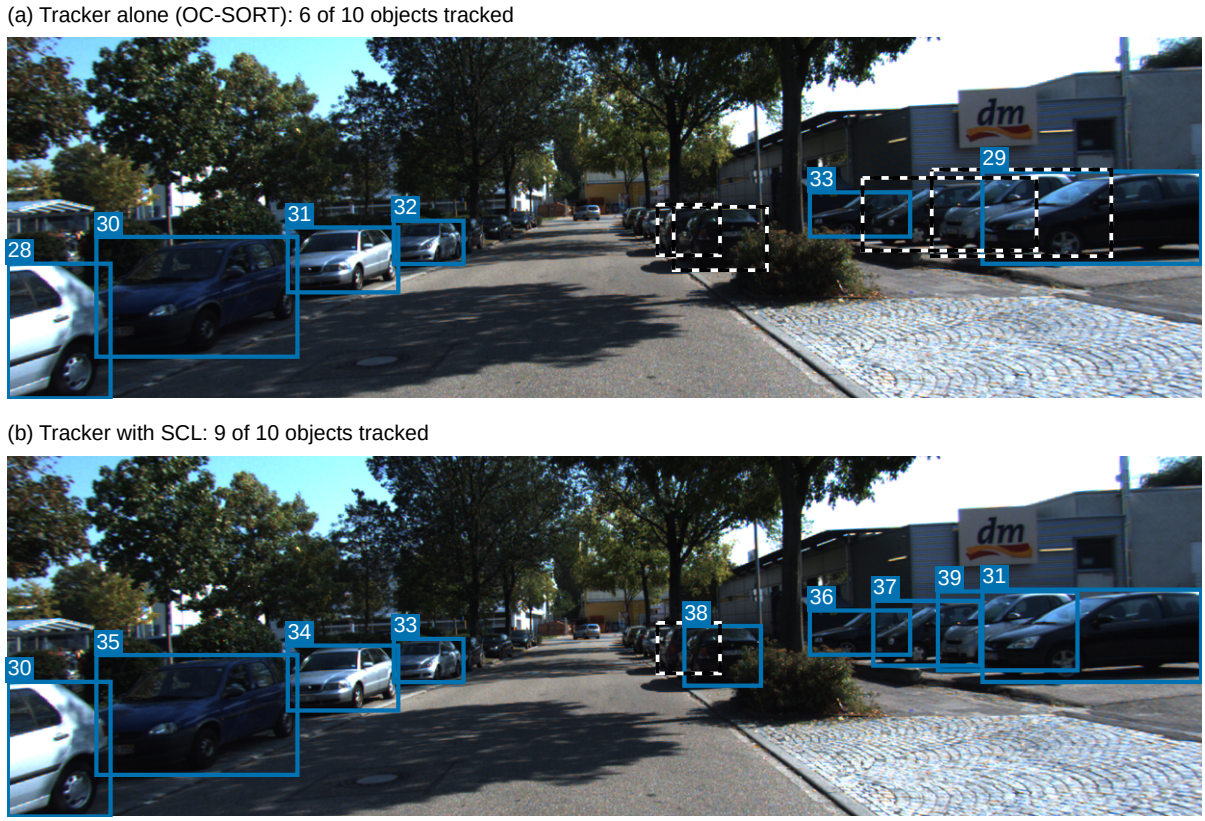


Fig. 7 Example frame from KITTI tracking training [16] (sequence 0001, frame 102), OC-SORT with YOLOv8n detections: (a) the tracker alone, (b) with SCL. Solid boxes with numbers: the tracks output for this frame and their IDs; dashed boxes: labelled cars and pedestrians that no output box covers. The sequence is the one with the median HOTA gain of the 21 sequences, and the frame is the first one in it with the largest gain in tracked objects

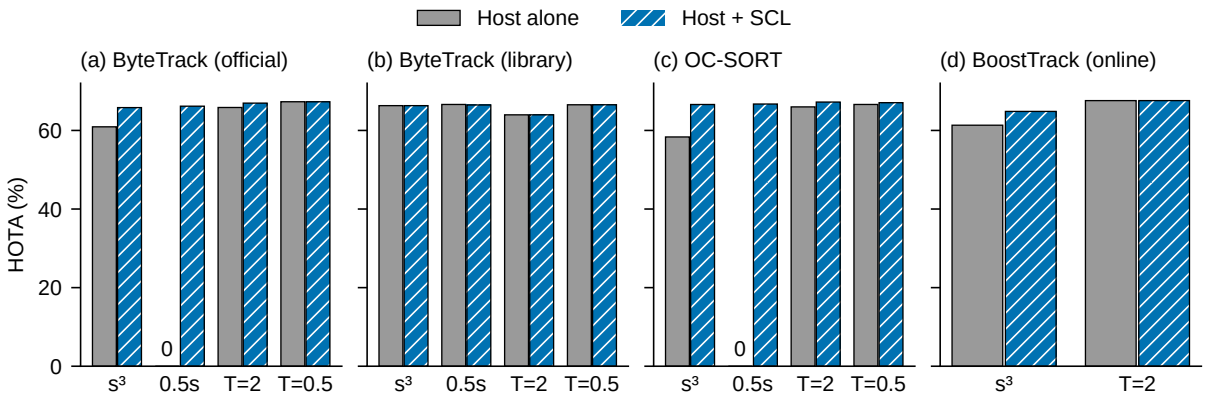


Fig. 8 Shifted detector scores on MOT17 val-half: HOTA of each published tracker with its published thresholds, alone and with SCL, after four monotone transforms of the detector scores (s^3, $0.5\,s$, and temperatures $T = 2$ and $T = 0.5$). (a) ByteTrack, official setting; (b) ByteTrack, library setting; (c) OC-SORT; (d) BoostTrack, online. Plain bars: the tracker alone; hatched bars: with SCL

5.4 Comparison with adaptive-threshold methods

A simpler way to follow the detector scores is to set the tracker's thresholds from them directly. Two such baselines were built on the same input and with the same window of the last 10 frames. They set the tracker's matching and birth thresholds in every frame to the first or the second Otsu split of the recent logits, t_1 or t_2, and pass the scores unchanged, without the mode decision, the new scores, or the rescue. Table 13 compares them with SCL on KITTI, for OC-SORT with YOLOv8n and for both trackers under halved scores. SCL reached the highest HOTA in all three cells, 2.2 to 8.6 points above the two baselines, and its paired-bootstrap intervals against the t_2 baseline lie above zero in every cell. Adaptive thresholds alone therefore do not explain the gains; the other parts of the layer add to them. With OC-SORT and YOLOv8n, SCL also had fewer identity switches than both baselines (170 against 1,545 and 250). Under halved scores, ByteTrack with SCL had more (501 against 273 and 275), while it missed far fewer objects (FN 7,750 against 12,357 with the t_2 baseline).

Comparison with a published method. ACT [11] sets the high and low thresholds of a two-stage tracker in every frame from the first quartile and the number of the current scores, using fixed score levels. It was run with its published parameters on the same detections as SCL, in three settings (Table 14): KITTI with halved scores, for ByteTrack and, with its high threshold as the single threshold, for OC-SORT; its own setting, MOT17 val-half with ByteTrack's official setting and the YOLOX-X detections; and the held-out UAVDT test split with ByteTrack. SCL reached higher HOTA than ACT in every row of Table 14. The difference was significant under halved scores (+2.01 and +10.06 points) and with YOLOv8n on UAVDT (+1.82 points, with +4.00 points of IDF1). In the setting of ACT itself, the two were close (+0.22 and +0.23 points). With RT-DETR-L on UAVDT, SCL reached a MOTA of 27.99 against 5.34 for ACT (difference +22.65, [+9.03, +47.21]): the host produced many false positives, and the fixed levels of ACT removed few of them (178,219 false positives, against 78,975 with SCL). SCL also had fewer identity switches than ACT on MOT17 and UAVDT, significantly with RT-DETR-L on UAVDT (418 fewer). Under halved scores on KITTI it had more (240 and 214 more), because it followed many more objects: it missed 7,750 objects against 12,326 for ACT with ByteTrack, and 13,547 against 21,887 with OC-SORT. The levels of ACT are fixed scores, so they do not move when the scores shift, while the splits of SCL are computed on the scores themselves.

Contribution of each part. Removing one part of the frozen SCL at a time shows where the gains come from, on the KITTI cells. Without the continuation rescue, the HOTA of OC-SORT fell by 3.34 points with YOLOv8n and by 1.16

Table 10 Baseline comparison of the development trackers on MOT17 val-half (published YOLOX-X detections). Paper: number reported by the tracker's authors. Host: the tracker alone, without SCL (our run of the published tracker). Δ: host with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42); bold intervals exclude zero. The floor is the lowest detector score emitted. IDS: identity switches, a count (lower is better); –: not reported by the authors.

Metric	Paper	Host	Δ [95% CI]
SparseTrack [32]			
HOTA	69.2	68.88	+0.06 [−0.01, +0.25]
MOTA	76.8	77.85	+0.08 [−0.03, +0.30]
IDF1	81.4	81.97	+0.16 [−0.02, +0.70]
IDS	–	124	+7 [0, +19]
BoostTrack, online [33]			
HOTA	68.37	68.49	0 (identical)
MOTA	75.56	75.50	0 (identical)
IDF1	81.35	81.41	0 (identical)
IDS	118	113	0 (identical)
BoostTrack, + GBI [33]			
HOTA	71.33	71.72	0 (identical)
MOTA	80.55	81.03	0 (identical)
IDF1	83.84	84.16	0 (identical)
IDS	106	97	0 (identical)
ByteTrack, floor 0.01 [5]			
HOTA	–	67.70	−0.014 [−0.058, +0.009]
MOTA	76.6	77.60	+0.06 [−0.15, +0.34]
IDF1	79.3	79.47	−0.031 [−0.135, +0.035]
IDS	–	214	+4 [−2, +13]
ByteTrack, floor 0.1 [5]			
HOTA	–	67.70	0 (identical)
MOTA	76.6	77.60	0 (identical)
IDF1	79.3	79.47	0 (identical)
IDS	–	214	0 (identical)
OC-SORT, floor 0.01 [6]			
HOTA	66.5	66.43	**+0.61 [+0.36, +1.24]**
MOTA	74.9	74.67	**+1.27 [+0.17, +3.01]**
IDF1	77.7	78.05	+0.66 [−0.25, +1.60]
IDS	–	211	**−8 [−15, −2]**
OC-SORT, floor 0.1 [6]			
HOTA	66.5	66.44	**+0.46 [+0.27, +1.09]**
MOTA	74.9	74.67	**+1.26 [+0.37, +3.34]**
IDF1	77.7	78.05	**+0.287 [+0.003, +0.838]**
IDS	–	213	−14 [−38, +6]

Table 11 Baseline comparison under a detector change on KITTI tracking training (official HOTA, car and pedestrian averaged). Host: the tracker alone, without SCL, with the same detections. Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better). BoT-SORT: KITTI sequence 0020 is left out of both runs, because the BoT-SORT code stopped with a numerical error in its Kalman update on it; the other 20 sequences are used.

Metric	Host	Δ [95% CI]
ByteTrack [5], *RT-DETR-L*		
HOTA	50.73	**+0.91 [+0.22, +1.55]**
MOTA	47.27	**+5.98 [+1.16, +9.94]**
IDF1	64.98	**+3.87 [+1.67, +5.06]**
IDS	686	**−297 [−530, −122]**
BoT-SORT [31], *RT-DETR-L* (20 sequences)		
HOTA	53.91	+0.61 [−0.39, +1.61]
MOTA	48.25	**+7.86 [+2.21, +13.44]**
IDF1	67.74	**+3.10 [+1.00, +4.57]**
IDS	476	**−226 [−369, −120]**
OC-SORT [6], *YOLOv8n*		
HOTA	36.27	**+7.91 [+5.47, +9.98]**
MOTA	33.99	**+9.40 [+1.10, +12.30]**
IDF1	50.46	**+9.60 [+5.61, +12.16]**
IDS	91	**+79 [+43, +125]**

points under halved scores. Using the clean mode in every frame, that is, without the mode decision, lowered HOTA by 0.90 points for ByteTrack with RT-DETR-L and by 2.40 and 2.44 points for the two trackers under halved scores. Without the new scores of the noisy mode, HOTA fell by 1.67 and 0.61 points under halved scores.

5.5 Frozen evaluation on external trackers

The external evaluation tests whether the frozen SCL helps trackers that were not seen during design. Table 15 gives the two predeclared external systems. PD-SORT [34] was reproduced exactly as in the authors' released output. With SCL, its HOTA rose by 0.61 points ($[+0.27, +1.65]$), its MOTA by 1.13 points ($[+0.27, +3.03]$), and its IDF1 by 0.82 points ($[+0.43, +1.94]$), with higher HOTA on 7 of 7 sequences (Fig. 6). FN fell by 1,291, FP rose by 681, and the identity switches stayed almost the same (131 against 129). PD-SORT is a single-stage tracker, and its gain comes from the same step as for OC-SORT; unlike OC-SORT, it was

Table 12 Baseline comparison under shifted detector scores on MOT17 val-half. The detector scores are transformed before the tracker, whose published thresholds stay fixed; T is a logit temperature, $\sigma(z/T)$. Host: the tracker alone, without SCL. Δ: with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42); bold intervals exclude zero. IDS: identity switches, a count (lower is better). A host that loses most detections also has few identity switches, because it follows few objects (for example 0 HOTA and 0 IDS under halved scores). Emission floor 0.01 (BoostTrack 0.1).

	HOTA (%)			IDS	
Transform	Host	+ SCL	Δ [95% CI]	Host	+ SCL
ByteTrack, official setting [5]					
s^3	60.92	65.81	**+4.89 [+2.60, +13.69]**	48	250
$0.5\,s$	0.00	66.17	**+66.17 [+49.32, +75.40]**	0	298
$T = 2$	65.84	66.95	**+1.11 [+0.35, +3.87]**	97	172
$T = 0.5$	67.30	67.29	−0.009 [−0.051, +0.005]	229	229
ByteTrack, library setting [5]					
s^3	66.31	66.31	−0.006 [−0.384, +0.383]	240	265
$0.5\,s$	66.61	66.48	−0.14 [−0.66, +0.14]	297	302
$T = 2$	63.97	63.97	0 (identical)	470	470
$T = 0.5$	66.53	66.52	−0.006 [−0.029, +0.001]	387	388
OC-SORT [6]					
s^3	58.34	66.60	**+8.26 [+5.86, +16.81]**	98	235
$0.5\,s$	0.00	66.72	**+66.72 [+50.32, +75.56]**	0	260
$T = 2$	65.99	67.22	**+1.24 [+0.52, +3.42]**	171	154
$T = 0.5$	66.61	67.06	+0.45 [−0.01, +0.97]	223	231
BoostTrack, online [33]					
s^3	61.33	64.84	**+3.51 [+0.85, +13.28]**	76	166
$T = 2$	67.59	67.59	0 (identical)	119	119

Table 13 HOTA and identity switches of SCL and of two simple adaptive-threshold baselines on KITTI tracking training (official HOTA, car and pedestrian averaged). Host: the tracker alone, without any layer. Adaptive t_1, t_2: the tracker's matching and birth thresholds are set in every frame to the first or second Otsu split of the logits of the last 10 frames, and the scores are passed unchanged. Δ: SCL minus the adaptive t_2 baseline, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better); a host with 0 HOTA follows no object, so it also has 0 IDS.

	HOTA (%)					IDS			
Cell	Host	Adapt. t_1	Adapt. t_2	SCL	Δ [95% CI]	Host	Adapt. t_1	Adapt. t_2	SCL
OC-SORT, YOLOv8n	36.27	39.97	41.09	44.17	**+3.09 [+2.00, +4.03]**	91	1545	250	170
ByteTrack, RT-DETR-L, halved scores	49.50	49.58	49.50	51.78	**+2.29 [+0.28, +4.96]**	279	273	275	501
OC-SORT, YOLOv8n, halved scores	0.00	37.68	43.56	46.30	**+2.74 [+1.32, +3.34]**	0	1975	492	308

Table 14 HOTA of ACT [11] and of SCL on the same detections, and the differences SCL minus ACT in HOTA, IDF1, and identity switches (IDS, a count; lower is better), with 95% paired-bootstrap interval below each value; bold intervals exclude zero. ACT: published parameters, no tuning. KITTI: official HOTA, car and pedestrian averaged; MOT17: val-half, ACT's own setting; UAVDT: test split, one vehicle class, ignored regions removed (Sect. 2.2).

Data	Detector	Tracker	ACT	SCL	ΔHOTA	ΔIDF1	ΔIDS
KITTI, halved scores	RT-DETR-L	ByteTrack	49.77	51.78	**+2.01** [**+0.01**, **+4.76**]	+1.70 [−1.13, +6.80]	**+240** [**+94**, **+401**]
KITTI, halved scores	YOLOv8n	OC-SORT	36.24	46.30	**+10.06** [**+6.88**, **+12.01**]	**+12.03** [**+6.85**, **+14.69**]	**+214** [**+141**, **+295**]
MOT17, floor 0.01	YOLOX-X	ByteTrack (official)	67.46	67.68	+0.22 [−0.43, +0.88]	+0.57 [−0.68, +2.12]	−18 [−62, +12]
MOT17, floor 0.1	YOLOX-X	ByteTrack (official)	67.46	67.70	+0.23 [−0.40, +0.90]	+0.60 [−0.55, +2.10]	−22 [−61, +4]
UAVDT test	YOLOv8n	ByteTrack	29.01	30.83	**+1.82** [**+0.65**, **+3.00**]	**+4.00** [**+1.85**, **+6.22**]	−185 [−480, +13]
UAVDT test	RT-DETR-L	ByteTrack	40.14	42.37	+2.23 [−0.97, +6.32]	+5.35 [−0.03, +12.47]	**−418** [**−718**, **−123**]

Table 15 Baseline comparison on the predeclared external systems (MOT17 val-half, authors' released detections), each evaluated once with the frozen SCL. Paper: number reported by the authors (for PD-SORT, their released output). Host: the tracker alone, without SCL (our run of the published tracker). Reproduction class against the authors' number (EXACT: $|\Delta\text{HOTA}| \leq 0.2$; CLOSE: ≤ 1.0 with a documented, untuned cause). Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better); –: not reported.

Metric	Paper	Host	Δ [95% CI]
PD-SORT [34], *EXACT*			
HOTA	68.01	68.01	**+0.61** [**+0.27**, **+1.65**]
MOTA	75.19	75.19	**+1.13** [**+0.27**, **+3.03**]
IDF1	81.03	81.03	**+0.82** [**+0.43**, **+1.94**]
IDS	–	129	+2 [−9, +13]
Hybrid-SORT [35], *CLOSE*			
HOTA	67.1	66.70	0 (identical)
MOTA	75.8	75.52	0 (identical)
IDF1	78.0	77.56	0 (identical)
IDS	–	259	0 (identical)

Table 16 Baseline comparison on the held-out UAVDT test split (20 aerial sequences, not used during design), evaluated once with the frozen SCL. Host: ByteTrack alone, without SCL, with its published setting. Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better).

Metric	Host	Δ [95% CI]
ByteTrack, RT-DETR-L [5, 20]		
HOTA	40.07	+2.30 [−1.05, +6.55]
MOTA	3.29	**+24.70** [**+10.06**, **+51.08**]
IDF1	48.59	**+5.74** [**+0.13**, **+13.11**]
IDS	1041	**−586** [**−951**, **−248**]
ByteTrack, YOLOv8n [5, 19]		
HOTA	29.05	**+1.77** [**+0.70**, **+2.94**]
MOTA	16.27	+1.02 [−1.06, +2.86]
IDF1	34.52	**+3.97** [**+1.88**, **+6.17**]
IDS	719	**−326** [**−691**, **−73**]

not seen during design. Hybrid-SORT [35], a two-stage tracker, gave identical output, as predicted before the run.

5.6 Frozen UAVDT transfer

The UAVDT transfer tests whether the frozen SCL still helps on aerial data that was not used during design, with none of its values changed. It uses the 20 test sequences of Sect. 2.1, and each cell was run once with the frozen configuration. Table 16 gives the results. SCL raised the HOTA of the YOLOv8n stream by 1.77 points ($[+0.70, +2.94]$) and its IDF1 by 3.97 points ($[+1.88, +6.17]$), and identity switches fell by 45% (from 719 to 393). With RT-DETR-L, the host produced many false positives (MOTA 3.29); with SCL, its MOTA rose by 24.70 points ($[+10.06, +51.08]$) and its IDF1 by 5.74 points ($[+0.13, +13.11]$): false positives fell by 58% and identity switches by 56% (from 1,041 to 455). HOTA did not fall with either detector.

6 Discussion

The score scale is part of the MOT system. The same detections and trackers give very different results when only the scale of the scores changes: halving the scores reduced ByteTrack (official setting) and OC-SORT from 67.70 and 66.43 HOTA to 0, and with RT-DETR-L on UAVDT, ByteTrack reached a MOTA of only 3.29 because of its many false positives. The fixed thresholds of a tracker hold only for the detector they were tuned for. The results therefore suggest that the score scale of the detector is a first-order variable of a tracking-by-detection system and deserves the same attention as the association method.

Self-limiting by design. In the clean mode the layer keeps the tracker's thresholds and scores, so a tracker with a low-score step and well-matched detections receives its normal input. This is the main difference from the earlier design AOB, which lowered the HOTA of the same strong trackers by 4.15 to 5.89 points (Table 8). The layer therefore does not need to know in advance whether a tracker needs help; it infers the operating mode from the score distribution.

Tracker dependence. Single-stage trackers drop every detection below one threshold, so a track can be lost when its object receives low scores for a few frames. The continuation rescue fills exactly these gaps, and OC-SORT and the external PD-SORT gained HOTA on 7 of 7 MOT17 sequences. Two-stage trackers already continue tracks with low-score boxes, so they keep their own accuracy with well-matched detections and gain when the detector is noisy, as ByteTrack and BoT-SORT did with RT-DETR-L. The observed gains are thus strongly associated with the host and with

how well its thresholds fit the detector scores, although scene and dataset properties, such as object density, motion, and occlusion, may also influence them.

Identity switches. The layer cut identity switches where it removed false boxes: by 43% and 47% for ByteTrack and BoT-SORT with RT-DETR-L on KITTI, by 45% and 56% on the held-out aerial data, and by 8 and 14 for OC-SORT on MOT17. Where it let a tracker follow many more objects, the count rose with the number of followed objects: for OC-SORT with YOLOv8n on KITTI, 79 more switches came with 5,310 fewer missed objects and 9.60 more IDF1 points. Identities were therefore kept better overall in this cell too.

Deployment. SCL needs no training, no GPU, and no access to the inside of the detector or the tracker. It reads four contract values from the tracker's published configuration and, at run time, the candidate list, one small grayscale image for the motion cue, and the tracks of the previous frame. On a CPU it takes 2.95 ms per frame, which is small next to the 42 ms of the detector, and the tracker itself ran faster with it, from 2.04 to 1.43 ms per frame (Table 7 and Fig. 3). The end-to-end time rose by 4.5%, so the layer adds a low overhead. Whether a whole pipeline meets a real-time target depends on the detector, the hardware, and the required frame rate; in this CPU test the full pipeline ran at 60.44 ms per frame, of which the detector took most. A new tracker is added through its published contract values and an adapter, without changes to the rules.

7 Conclusion

This paper presented SCL, a training-free layer that sits between an unchanged detector and an unchanged tracker and adapts the tracker's input to the detector's scores, using only the scores of recent frames and the tracks of the previous frame. Nested Otsu splits of the recent logits decide whether the scores are clean or noisy. On clean scores the layer keeps the tracker's settings; on noisy scores it chooses the candidates, gives them new scores, and sets the thresholds; and a continuation rescue gives single-stage trackers the step they lack. A host contract of four published settings lets one fixed configuration serve many trackers. The layer was developed step by step from AOB and was then frozen before any external test.

One frozen configuration was tested on seven published trackers, four detection sources, and three datasets. With halved detector scores, it restored two trackers from 0 to about 66 HOTA. It raised the HOTA of both single-stage trackers, including the external PD-SORT, on every MOT17 sequence. With RT-DETR-L on KITTI, it raised the MOTA of ByteTrack and BoT-SORT by 5.98 and 7.86 points and cut their identity switches by 43% and 47%, and on held-out aerial data it raised MOTA by 24.70 points and cut identity switches by up to 56%. It reached higher HOTA than simple adaptive thresholds on the same scores and than the published ACT method under halved scores and on held-out aerial data, and kept the tested well-matched two-stage trackers within 0.06 HOTA points of their own accuracy on MOT17. No tested cell showed a statistically significant HOTA loss, and the largest observed decrease was 0.14 points; this does not prove that no loss can occur with other trackers or data. The layer took 2.95 ms per frame on a CPU.

Overall, the results indicate that the score scale of the detector should be treated as part of the tracking system rather than as a fixed setting of the tracker. Future work will test the layer with more detectors, trackers, and datasets.

Statements and Declarations

Funding. The authors did not receive support from any organization for the submitted work.

Competing interests. The authors have no competing interests to declare that are relevant to the content of this article.

Ethics approval and consent to participate. Not applicable. The study uses only public benchmark datasets and involves no human participants or animals.

Consent for publication. Not applicable.

Data availability. The datasets analysed in this study are publicly available: MOT17 (https://motchallenge.net), KITTI tracking (https://www.cvlibs.net/datasets/kitti), and UAVDT (https://sites.google.com/view/grli-uavdt). The detections released by the authors of the evaluated

trackers were obtained from their public repositories. The per-sequence results and bootstrap outputs generated in this study are available from the corresponding author on reasonable request.

Materials availability. Not applicable.

Code availability. The code of the layer, its frozen configuration, the tracker adapters, and the evaluation scripts are available from the corresponding author on reasonable request.

Author contributions. Ahmed Gouda Ismail: conceptualization, methodology, software, investigation, formal analysis, and writing of the original draft. Mohamed S. Mohamed and Tarek Ahmed Mahmoud: supervision, and review and editing of the manuscript. All authors read and approved the final manuscript.

References

[1] Zhu, P., Wen, L., Du, D., Bian, X., Fan, H., Hu, Q., Ling, H.: Detection and Tracking Meet Drones Challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7380–7399 (2022) https://doi.org/10.1109/tpami.2021.3119563

[2] Wu, X., Li, W., Hong, D., Tao, R., Du, Q.: Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A survey. IEEE Geoscience and Remote Sensing Magazine **10**(1), 91–124 (2022) https://doi.org/10.1109/mgrs.2021.3115137

[3] Adžemović, M.: Deep learning-based multi-object tracking: a comprehensive survey from foundations to state-of-the-art. The Visual Computer **42**(9), 368 (2026) https://doi.org/10.1007/s00371-026-04607-7

[4] Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: Simple online and realtime tracking. In: 2016 IEEE International Conference on Image Processing (ICIP), pp. 3464–3468 (2016). https://doi.org/10.1109/icip.2016.7533003

[5] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., Wang, X.: ByteTrack: Multi-object Tracking by Associating Every Detection Box. In: Computer Vision – ECCV 2022. Lecture Notes in Computer Science, pp. 1–21 (2022). https://doi.org/10.1007/978-3-031-20047-2_1

[6] Cao, J., Pang, J., Weng, X., Khirodkar, R., Kitani, K.: Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9686–9696 (2023). https://doi.org/10.1109/cvpr52729.2023.00934

[7] Küppers, F., Kronenberger, J., Shantia, A., Haselhoff, A.: Multivariate Confidence Calibration for Object Detection. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1322–1330 (2020). https://doi.org/10.1109/cvprw50498.2020.00171

[8] Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., Leibe, B.: HOTA: A Higher Order Metric for Evaluating Multi-object Tracking. International Journal of Computer Vision **129**(2), 548–578 (2021) https://doi.org/10.1007/s11263-020-01375-2

[9] Ma, L.V., Hussain, M.I., Park, J., Kim, J., Jeon, M.: Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking. In: 2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS), pp. 370–374 (2023). https://doi.org/10.1109/iccais59597.2023.10382403

[10] Shim, K., Ko, K., Hwang, J., Kim, C.: AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking. In: 2024 IEEE International Conference on Image Processing (ICIP), pp. 2222–2228 (2024). https://doi.org/10.1109/icip51287.2024.10648139

[11] Gwon, D.U., Lee, W.I.: Adaptive Threshold Control Scheme Based on Detection Confidence Distribution for Data Association in Multi-Object Tracking. Electronics **15**(17), 4047 (2026) https://doi.org/10.3390/electronics15174047

[12] Han, Y., Huang, G., Song, S., Yang, L.,

Wang, H., Wang, Y.: Dynamic Neural Networks: A Survey. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(11), 7436–7456 (2022) https://doi.org/10.1109/tpami.2021.3117837

[13] Jiang, J., Ananthanarayanan, G., Bodik, P., Sen, S., Stoica, I.: Chameleon: Scalable Adaptation of Video Analytics. In: Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication, pp. 253–266 (2018). https://doi.org/10.1145/3230543.3230574

[14] Otsu, N.: A Threshold Selection Method from Gray-Level Histograms. IEEE Transactions on Systems, Man, and Cybernetics **9**(1), 62–66 (1979) https://doi.org/10.1109/tsmc.1979.4310076

[15] Milan, A., Leal-Taixe, L., Reid, I., Roth, S., Schindler, K.: MOT16: A Benchmark for Multi-Object Tracking. arXiv preprint arXiv:1603.00831 (2016). https://doi.org/10.48550/arXiv.1603.00831

[16] Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? The KITTI vision benchmark suite. In: 2012 IEEE Conference on Computer Vision and Pattern Recognition, pp. 3354–3361 (2012). https://doi.org/10.1109/cvpr.2012.6248074

[17] Du, D., Qi, Y., Yu, H., Yang, Y., Duan, K., Li, G., Zhang, W., Huang, Q., Tian, Q.: The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking. In: Computer Vision – ECCV 2018. Lecture Notes in Computer Science, pp. 375–391. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-01249-6_23

[18] Ge, Z., Liu, S., Wang, F., Li, Z., Sun, J.: YOLOX: Exceeding YOLO Series in 2021. arXiv preprint arXiv:2107.08430 (2021). https://doi.org/10.48550/arXiv.2107.08430

[19] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics. Version 8.0.0 (2023)

[20] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs Beat YOLOs on Real-time Object Detection. In: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024). https://doi.org/10.1109/cvpr52733.2024.01605

[21] Dendorfer, P., Ošep, A., Milan, A., Schindler, K., Cremers, D., Reid, I., Roth, S., Leal-Taixé, L.: MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking. International Journal of Computer Vision **129**(4), 845–881 (2021) https://doi.org/10.1007/s11263-020-01393-0

[22] Bernardin, K., Stiefelhagen, R.: Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics. EURASIP Journal on Image and Video Processing **2008**, 1–10 (2008) https://doi.org/10.1155/2008/246309

[23] Ristani, E., Solera, F., Zou, R., Cucchiara, R., Tomasi, C.: Performance Measures and a Data Set for Multi-target, Multi-camera Tracking. In: Computer Vision – ECCV 2016 Workshops. Lecture Notes in Computer Science, pp. 17–35 (2016). https://doi.org/10.1007/978-3-319-48881-3_2

[24] Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap, (1993). https://doi.org/10.1007/978-1-4899-4541-9

[25] Li, Y., Liu, Y., Zhou, C., Xu, D., Tao, W.: A lightweight scheme of deep appearance extraction for robust online multi-object tracking. The Visual Computer **40**(3), 2049–2065 (2024) https://doi.org/10.1007/s00371-023-02901-2

[26] Bao, Y., Yu, Y., Qi, Y., Wang, Z.: Multiple object tracking with adaptive multi-features fusion and improved learnable graph matching. The Visual Computer **40**(4), 2279–2292 (2024) https://doi.org/10.1007/s00371-023-02916-9

[27] Zhang, Y., Liang, Y., Wang, J., Zhu, H., Wang, Z.: Enhanced multi-object tracking via embedded graph matching and differentiable Sinkhorn assignment: addressing

challenges in occlusion and varying object appearances. The Visual Computer **41**(8), 6067–6085 (2025) https://doi.org/10.1007/s00371-024-03772-x

[28] Ding, Y., Wang, Y., Wang, L., Zhang, B., Li, J., Guo, L., Yang, Z., Liang, W.: Enhancing multi-object tracking efficiency through result-guided feature extraction and query filtering. The Visual Computer **41**(11), 8153–8167 (2025) https://doi.org/10.1007/s00371-025-03861-5

[29] Liang, Z., Xiao, G., Hu, J., Wang, J., Ding, C.: MotionTrack: rethinking the motion cue for multiple object tracking in USV videos. The Visual Computer **40**(4), 2761–2773 (2024) https://doi.org/10.1007/s00371-023-02983-y

[30] Sun, Z., Lv, Z., Xiong, M., Hu, W., Wei, G., Zhou, J., He, T., Meng, Q.: Online visual multi-object tracking via blind super-resolution and clustering learning. The Visual Computer **42**(4), 189 (2026) https://doi.org/10.1007/s00371-026-04382-5

[31] Aharon, N., Orfaig, R., Bobrovsky, B.-Z.: BoT-SORT: Robust Associations Multi-Pedestrian Tracking. arXiv preprint arXiv:2206.14651 (2022). https://doi.org/10.48550/arXiv.2206.14651

[32] Liu, Z., Wang, X., Wang, C., Liu, W., Bai, X.: SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth. IEEE Transactions on Circuits and Systems for Video Technology **35**(5), 4870–4882 (2025) https://doi.org/10.1109/tcsvt.2024.3524670

[33] Stanojevic, V.D., Todorovic, B.T.: BoostTrack: boosting the similarity measure and detection confidence for improved multiple object tracking. Machine Vision and Applications **35**(3) (2024) https://doi.org/10.1007/s00138-024-01531-5

[34] Wang, Y., Zhang, D., Li, R., Zheng, Z., Li, M.: PD-SORT: Occlusion-Robust Multi-Object Tracking Using Pseudo-Depth Cues. IEEE Transactions on Consumer Electronics **71**(1), 165–177 (2025) https://doi.org/10.1109/tce.2025.3541839

[35] Yang, M., Han, G., Yan, B., Zhang, W., Qi, J., Lu, H., Wang, D.: Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking. Proceedings of the AAAI Conference on Artificial Intelligence **38**(7), 6504–6512 (2024) https://doi.org/10.1609/aaai.v38i7.28471

[36] Jung, H., Kang, S., Kim, T., Kim, H.: ConfTrack: Kalman Filter-based Multi-Person Tracking by Utilizing Confidence Score of Detection Box. In: 2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 6569–6578 (2024). https://doi.org/10.1109/wacv57701.2024.00645

[37] Solano-Carrillo, E., Sattler, F., Alex, A., Klein, A., Costa, B.P., Rodriguez, A.B., Stoppe, J.: UTrack: Multi-object Tracking with Uncertain Detections. In: Computer Vision – ECCV 2024 Workshops. Lecture Notes in Computer Science, pp. 219–236. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-91585-7_14

[38] Kim, S., Lee, S., Cho, M.: PlugTrack: Multi-Perceptive Motion Analysis for Adaptive Fusion in Multi-Object Tracking. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 40, pp. 5709–5716 (2026). https://doi.org/10.1609/aaai.v40i7.37491

[39] Wang, C.-Y., Yeh, I.-H., Liao, H.-Y.M.: YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. In: Computer Vision – ECCV 2024. Lecture Notes in Computer Science, pp. 1–21 (2024). https://doi.org/10.1007/978-3-031-72751-1_1

[40] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., Zagoruyko, S.: End-to-End Object Detection with Transformers. In: Computer Vision – ECCV 2020. Lecture Notes in Computer Science, pp. 213–229 (2020). https://doi.org/10.1007/978-3-030-58452-8_13

[41] Reddy, B.S., Chatterji, B.N.: An FFT-based technique for translation, rotation, and scale-invariant image registration. IEEE Transactions on Image Processing **5**(8), 1266–1271 (1996) https://doi.org/10.1109/83.506761